

QuikSkill LMS

Backend System Documentation & Migration Reference

Complete Data Model · API Routes · Architecture · Features

Current Stack: **NestJS 10** + **MongoDB (Mongoose)** + **Redis/Bull** + **AWS S3**

Target Stack: **Next.js** + **PostgreSQL**

Generated: 5 June 2026 | Document 1 of 2 (Backend)

Table of Contents

1. System Overview & Tech Stack
2. Architecture & Multi-Tenancy Model
3. Authentication, Roles & Authorization (RBAC)
4. Cross-Cutting Infrastructure (Middleware, WebSockets, Queues, Cron, Storage, Integrations)
5. Environment Variables
6. Complete Data Model — All Collections / Future Postgres Tables
 - 5.1 Identity & Tenancy 5.2 Courses & Content 5.3 Progress, Assessments & Certificates
 - 5.4 School Operations 5.5 Communications 5.6 Exams & Proctoring 5.7 Supporting Tables
7. Complete API Route Reference (by Module)
8. MongoDB → PostgreSQL Migration Guidance

1. System Overview & Tech Stack

QuikSkill LMS is a multi-tenant Learning Management System serving two tenant archetypes from a single codebase: **Corporate** (employee training, compliance, manager hierarchy) and **School** (batches, attendance, credits, payouts, parents, live classes, proctored exams). The backend is a modular NestJS monolith exposing a REST API plus two Socket.IO gateways.

Current Technology Stack

Layer	Technology	Notes / Migration Target
Runtime / Framework	Node.js + NestJS 10 (TypeScript)	Becomes Next.js (API routes / route handlers, or a retained Nest service)
Database	MongoDB 8 via Mongoose 8	→ PostgreSQL (Prisma/Drizzle/TypeORM recommended)
Cache / Queue	Redis + Bull (@nestjs/bull)	Retain Redis; Bull → BullMQ or pg-boss
Object Storage	AWS S3 (presigned PUT/GET, TUS resumable)	Retain S3
Auth	JWT (passport-jwt) + Google/Microsoft OAuth + OTP	Retain; or NextAuth/Auth.js
Realtime	Socket.IO (messages + exam gateways)	Retain (separate WS server in Next.js)
Scheduling	@nestjs/schedule cron + Bull delayed jobs	Cron worker / serverless cron
Email	Nodemailer (SMTP, Office365) + Resend	Retain
SMS / Voice	Twilio	Retain
AI	OpenAI (Whisper transcription / captions)	Retain
Video	Zoom / Google Meet / Jitsi providers	Retain
Docs	PDFKit / html-pdf-node (certificates, reports)	Retain

Scale of the system: 59 NestJS modules · 48 REST controllers · 55 Mongoose schemas (collections) · 62 services · 2 WebSocket gateways · 7 user roles. Default API port **3020** . Body size limit 150 MB (large media uploads).

Module Inventory (59 modules)

Core: features, tenants, auth, users, courses, assessments, scorm, upload, shared-content, seed, email-templates, progress, certificates, audit, course-assignments, compliance, manager, learner, player, jobs, groups, bookmarks, discussions, counters. School: batches, scheduling, credits, attendance, homework, payouts, meetings, messages, analytics. Enhancements: twilio, escalation, student-reminder, reminders, course-assignment-reminders, non-teaching-work, teacher-level, teacher-availability, demo-analytics, privacy, i18n, transcription. Academic/New: tutoring-requests, academic-calendar, gradebook, consent, academic-config. Exams: question-bank, exams, exam-sessions, proctoring, quiz-proctoring, bulk-upload.

2. Architecture & Multi-Tenancy Model

Every business record carries a `tenantId`. A request's tenant is resolved from the JWT payload, the subdomain (e.g. `company1.localhost`), or an `X-Tenant-Key` header (`TenantResolverMiddleware`). `TenantGuard` blocks cross-tenant reads/writes; `SUPER_ADMIN` bypasses tenant scoping to administer all organizations.

- **Tenant types:** `corporate` | `school` — drives which feature flags & modules are active (stored on `Tenant.tenantType` + `Tenant.featureConfig`).
- **Feature flags:** per-tenant `featureConfig` object toggles courses, SCORM, compliance, manager reports, batches, attendance, homework, credits, payouts, video classes, parent portal, messaging, certificates, analytics, approval workflow.
- **Content sharing:** `SUPER_ADMIN` authors **master courses / certificate templates** in a global library and pushes copies to selected tenants (`shared-content` , `selectedTenants`).
- **Approval workflow:** `SUB_ADMIN` / tenant-created courses & certificates can require `TENANT_ADMIN` or `SUPER_ADMIN` approval (`approvalStatus` fields).

3. Authentication, Roles & Authorization (RBAC)

3.1 User Roles (UserRole enum)

Role	Context	Scope
SUPER_ADMIN	Platform	All tenants; master library, approvals, system health, audit
TENANT_ADMIN	Corporate & School	Full admin of one tenant
SUB_ADMIN	Corporate & School	Delegated admin; some actions need TENANT_ADMIN approval
MANAGER	Corporate	Team lead — oversees direct reports' learning
TEACHER	School	Instructor — classes, attendance, homework, payouts
PARENT	School	Guardian — child schedule, homework, credits, live classes
LEARNER	Corporate & School	Student / employee — courses, exams, certificates

Users may hold a `secondaryRole` ; the `RolesGuard` checks both primary and secondary role against an endpoint's `@Roles(...)` list.

3.2 Strategies, Guards & Flows

Component	File	Responsibility
JWT Strategy	auth/strategies/jwt.strategy.ts	Parse <code>Authorization: Bearer</code> ; secret <code>JWT_SECRET</code> ; re-validates user is active; enriches req.user with role, tenantId, tenantType, secondaryRole
Google Strategy	strategies/google.strategy.ts	OAuth2 via <code>GOOGLE_CLIENT_ID/SECRET</code>
Microsoft Strategy	strategies/microsoft.strategy.ts	OAuth2 via Microsoft
Local Strategy	strategies/local.strategy.ts	Email + password login
JwtAuthGuard	guards/jwt-auth.guard.ts	Validates JWT; honors <code>@Public()</code>
RolesGuard	guards/roles.guard.ts	Enforces <code>@Roles()</code> ; ForbiddenException on mismatch
TenantGuard	guards/tenant.guard.ts	Blocks cross-tenant access (SUPER_ADMIN bypass)
Google/Microsoft/LocalAuthGuard	guards/*.guard.ts	Passport guards for each strategy
FeatureGuard	guards/feature.guard.ts	Feature-flag gated endpoints

OTP & Password flows: `POST /auth/forgot-password` issues a 4-digit OTP (stored in `otp` with a 10-minute TTL index) → `POST /auth/verify-otp` → `POST /auth/reset-password`. New users receive a `passwordSetupToken` (`/auth/validate-setup-token` → `/auth/setup-password`); `mustChangePassword` forces a change on first login. Admins can force-reset via `/auth/admin-reset-password`.

4. Cross-Cutting Infrastructure

4.1 Global Middleware, Pipes & Filters (main.ts / app.module.ts)

Component	Behaviour
CORS	<code>origin: '*'</code> ; methods GET/HEAD/PUT/PATCH/POST/DELETE/OPTIONS; headers Content-Type, Authorization
Body Parser	JSON & urlencoded limit 150 MB
LowercaseEmailsMiddleware	Recursively lowercases+trims every <code>*email*</code> field before validation
ValidationPipe (global)	<code>whitelist</code> , <code>forbidNonWhitelisted</code> , <code>transform</code> — strips/blocks unknown props, auto-casts DTOs
AllExceptionsFilter	Standard error JSON: statusCode, timestamp, path, method, requestId, message, error, validationErrors; never leaks stack in prod
RequestLoggingMiddleware	Adds <code>X-Request-Id</code> UUID; logs method/url/status/duration with tenantId+userId; Winston daily-rotate files
TenantResolverMiddleware	Resolves tenant from subdomain / <code>X-Tenant-Key</code>
Privacy interceptor	Strips sensitive fields & records to <code>privacy-audit-log</code>

4.2 WebSocket Gateways (Socket.IO)

Gateway	Namespace	Auth	Events / Behaviour
MessagesGateway	<code>/messages</code>	JWT in handshake	Send/receive messages, typing indicators, online presence, notifications. Rate limit: 20 msgs / 60s / user
ExamGateway	<code>/exams</code>	JWT in handshake	<code>joinExam</code> / <code>leaveExam</code> rooms (<code>exam:<sessionId></code>); emits <code>timerSync</code> (30s), <code>examWarning</code> (5min/1min), <code>forceSubmit</code> on timeout

4.3 Background Jobs — Cron Schedules

Cron	Service	Purpose
<code>0 9 * * *</code> (daily 09:00 UTC)	DeadlineRemindersService	Remind learners of assignments due within 3 days
<code>30 9 * * *</code> (daily 09:30 UTC)	DeadlineRemindersService	Remind learners of overdue assignments
<code>0 7 * * *</code> (daily 07:00 UTC)	TeacherLatenessRemindersService	Detect teachers late in last 2 sessions → notify (logged in <code>teacher-lateness-reminder-log</code>)
<code>* * * * *</code> (every minute)	StudentReminderService	Find in-progress classes past grace period → queue calls to absent students

4.4 Background Jobs — Bull Queues (Redis)

Queue	Processor	Purpose
<code>exam-auto-submit</code>	ExamAutoSubmitProcessor	Auto-submit an exam session when its server deadline expires
<code>reminders</code>	RemindersProcessor	Class reminders at -30min (email), -10min (email+SMS), -5min (SMS) with join link
<code>course-assignment-reminders</code>	CourseAssignmentRemindersProcessor	Immediate "assigned" email + day-10 & day-20 follow-ups (skips if completed)
<code>escalation</code>	EscalationProcessor	Call teacher up to 3× when absent from a live class; notify admin
<code>student-reminder</code>	StudentReminderProcessor	Call absent students up to 3× (guardianContact → phone) via Twilio

4.5 File Storage (AWS S3)

- **Presigned PUT** (`UploadService`): browser uploads directly to S3. Key format `tenants/{tenantId}/uploads/{uuid}-{fileName}`; returns `uploadUrl`, `fileKey`, `fileUrl`.

- **Presigned GET** (`S3PresignedService`): time-limited read URLs (default 3600s); handles full URLs, bare keys, data URLs.
- **TUS resumable uploads** (`@tus/server`) for large media at `POST /upload/tus` .
- **SCORM**: zip parse/extract/validate (manifest.xml) up to 500 MB.
- **Storage quotas**: per-tenant `storageLimit` (GB); usage tracked & surfaced via `/tenants/usage` , `/audit/storage/*` .

4.6 External Integrations

Integration	Use	Key Env Vars
Nodemailer (SMTP)	Transactional email, PDF attachments	SMTP_HOST, SMTP_PORT, SMTP_USER, SMTP_PASS
Resend	Alt email provider	RESEND_API_KEY
Twilio	SMS & voice (reminders, escalation); +91 normalization	TWILIO_ACCOUNT_SID, TWILIO_AUTH_TOKEN, TWILIO_PHONE_NUMBER
OpenAI Whisper	Video caption/transcription (.vtt → S3), max 25 MB	OPENAI_API_KEY
Google APIs	Google Meet provider, OAuth	GOOGLE_CLIENT_ID, GOOGLE_CLIENT_SECRET
Zoom / Jitsi / Teams	Meeting providers (per-tenant <code>videoConfig</code>)	Stored per-tenant in DB

5. Environment Variables

```
# Database
MONGODB_URI=mongodb://localhost:27017/quikskill-lms    # → DATABASE_URL (Postgres) after migration
# Redis (Bull queues)
REDIS_URL=redis://localhost:6379
# JWT
JWT_SECRET=...          (default fallback: quikskill-lms-dev)
# AWS S3
AWS_ACCESS_KEY_ID=...   AWS_SECRET_ACCESS_KEY=...   AWS_REGION=ap-south-1   AWS_S3_BUCKET=quik-lms
# Email
SMTP_HOST=smtp.office365.com  SMTP_PORT=587  SMTP_USER=...  SMTP_PASS=...  RESEND_API_KEY=...
# OAuth
GOOGLE_CLIENT_ID=...  GOOGLE_CLIENT_SECRET=...  GOOGLE_CALLBACK_URL=...
MICROSOFT_CLIENT_ID=...  MICROSOFT_CLIENT_SECRET=...  MICROSOFT_CALLBACK_URL=...
# Twilio
TWILIO_ACCOUNT_SID=...  TWILIO_AUTH_TOKEN=...  TWILIO_PHONE_NUMBER=+1...
# OpenAI
OPENAI_API_KEY=sk-...
# App
PORT=3020  NODE_ENV=development  BACKEND_URL=...  ENCRYPTION_KEY=... (32-char, for aiApiKey)
```

6. Complete Data Model — All Collections / Future Postgres Tables

Every collection below maps to a PostgreSQL table. Mongo `ObjectId` → `uuid` or `bigint` PK; `ObjectId ref→X` → foreign key; embedded **[subdoc]** arrays → either a child table (1-to-many) or a `jsonb` column; `Map/Object` → `jsonb`; enums → Postgres `enum` or `text` +CHECK. All business tables include `tenantId` (FK to `tenants`) and Mongoose `timestamps` give `createdAt` / `updatedAt`.

55 collections total. Sub-documents (arrays of objects) are flagged inline — these are the main normalization decisions for the relational schema.

6.1 Identity & Tenancy

users — every person on the platform (all 7 roles)

Field	Type	Req	Notes
email	String	Y	lowercase, trimmed, indexed; unique per tenant
password	String	N	null for OAuth users (bcrypt hash)
firstName / lastName	String	Y	
role	enum	Y	SUPER_ADMIN, TENANT_ADMIN, SUB_ADMIN, MANAGER, TEACHER, LEARNER, PARENT (default LEARNER)
secondaryRole	enum	N	multi-role users
tenantId	String	N	owning tenant
managerId	ObjectId→User	N	corporate hierarchy
isActive	Boolean	Y	default true
parentIds	[ObjectId→User]	N	LEARNER → linked parents
parentEmail	String	N	deferred parent linking during bulk upload
childrenIds	[ObjectId→User]	N	PARENT → linked students
guardianContact / phone	String	N	SMS/call reminders
guardianRelation	String	N	father/mother/guardian
grade / section	String	N	student class & section
subjects	[String]	N	teacher subjects
ratePerClass / ratePerHour / monthlyPayout	Number	N	teacher pay
rateType	String	N	per_class/per_hour/monthly/hybrid
qualification	String	N	PGT/TGT/PRT/NTT/Other
employeeId / studentId / parentCode	String	N	unique sparse per tenant
dateOfBirth	Date	N	
availableSlots	[subdoc]	N	{dayOfWeek, startTime, endTime} — teacher availability
maxSlotsPerWeek	Number	N	
tutoringEnabled / tutoringCreditCost	Boolean / Number	N	1:1 tutoring
classesCompleted / classesMissed / classesCancelled / classesRescheduledAndCompleted	Number	Y	teacher counters (default 0)
punctualityScore	Number	N	0–100
provider / providerId	String	N	google/microsoft/local OAuth
profilePicture	String	N	S3 URL
aiApiKey	String	N	encrypted, select:false

Field	Type	Req	Notes
mustChangePassword	Boolean	Y	default true
passwordSetupToken / passwordSetupTokenExpiry	String / Date	N	first-time setup
activeSessionId	String	N	single-session enforcement
currentStreak / totalPoints / lastActivityDate	Number/Date	Y/N	gamification
preferredLanguage / timezone	String	Y/N	default 'en'
notificationPreferences	Object	Y	email/class/homework/credit/payout toggles → jsonb

Indexes: (tenantId,totalPoints!), (tenantId,currentStreak!), (tenantId,role), (tenantId,grade), (parentIds), (childrenIds), unique-sparse (tenantId,studentId)/(tenantId,employeeId)/(tenantId,parentCode), (tenantId,email), (tenantId,parentEmail).

otps — password-reset / 2FA codes

Field	Type	Req	Notes
email	String	Y	indexed
otp	String	Y	4-digit
expiresAt	Date	Y	TTL index 600s
isUsed	Boolean	Y	default false

tenants — organizations (corporate or school)

Field	Type	Req	Notes
name / subdomain / tenantKey	String	Y	all unique
tenantType	enum	Y	corporate school (default corporate)
status	enum	Y	Active Paused Trial
gstNumber	String	N	unique sparse
dbConnectionString	String	N	optional dedicated DB
orgName / fullAddress / country / officialPhone / officialEmail / website	String	Y*	org profile
contact* (firstName, middleName, lastName, phone, email, roleInOrganization)	String	Y*	primary contact
billing* (firstName, middleName, lastName, address)	String	Y*	billing contact
featureConfig	Object→jsonb	Y	enableCourses, enableScorm, enableCompliance, enableManagerReports, enableSelfEnrollment, enableBatches, enableAttendance, enableHomework, enableCredits, enablePayouts, enableVideoClasses, enableParentPortal, enableMessaging, enableCertificates, enableAnalytics, approvalWorkflowEnabled
schoolConfig	Object→jsonb	N	academicYearStartMonth, gradeLevels[], subjects[], defaultCreditExpiryMonths, defaultTeacherRatePerClass
corporateConfig	Object→jsonb	N	enableComplianceTracking, complianceDueDays, enableTeamHierarchy, enableSelfEnrollment, departmentList[]
creditConfig	Object→jsonb	N	expiry policy, thresholds, zeroCreditPolicy, packages[]
payoutConfig	Object→jsonb	N	default rates, payoutGenerationDay, autoApprove, paymentMethods[]
videoConfig	Object→jsonb	N	provider, credentials, settings, requireParentConsent
enhancementConfig	Object→jsonb	N	escalation, studentReminder, teacherLevel, demoTrial tuning
timezone / defaultLanguage / enabledLanguages / locale / currency / localeSettings	mixed	Y	i18n & locale
storageLimit	Number	Y	GB (default 2)
logoUrl / faviconUrl / primaryColor / secondaryColor	String	N/Y	branding (#3B82F6 / #1E40AF)

Field	Type	Req	Notes
auth0OrganizationId / auth0UserId / loginUrl	String	N	SSO

counters — sequential ID generator

Field	Type	Notes
tenantId	ObjectId→Tenant	unique (tenantId,type)
type	String	teacher student parent
seq	Number	current value (default 0)

groups — user groups for bulk course assignment

Field	Type	Notes
tenantId	ObjectId→Tenant	unique (tenantId,name)
name / description	String	
memberIds	[ObjectId→User]	→ join table group_members
createdBy	ObjectId→User	

6.2 Courses & Content

courses

Field	Type	Req	Notes
tenantId	ObjectId	N	null for master courses (sparse idx)
title / category / description	String	Y/N	
authorId	ObjectId→User	Y	
modules	[ObjectId→Module]	Y	ordered module refs
status	enum	Y	Draft Published Archived
thumbnailUrl	String	N	S3
isMaster	Boolean	Y	global library course
selectedTenants	[ObjectId→Tenant]	N	distribution targets
isFeatured / version	Boolean / Number	Y	
prerequisites	[ObjectId→Course]	Y	

modules (contains embedded Lesson subdocs)

Field	Type	Req	Notes
tenantId	ObjectId	N	sparse
courseId	ObjectId→Course	Y	
title / description	String	Y/N	
orderIndex	Number	Y	
lessons	[Lesson subdoc]	Y	see below → child table lessons
assessmentId	ObjectId→Assessment	N	module quiz

Lesson subdoc: title, type(Video/PDF/PPT/SCORM/Text/Quiz/Audio/Document/Slides), contentUrl, orderIndex, isMaster, description, duration, fileSize, captions[{language,label,url}], quiz[questions[{id,text,options,questionType,correctAnswer}], passingScore].

assessments (embedded Question subdocs) & **quiz-attempts**

Collection	Field	Type	Notes
assessments	tenantId	ObjectId→Tenant	
	moduleId	ObjectId→Module	
	title	String	
	questions	[Question subdoc]	text, type(MCQ/True-False), options[], correctAnswerIndex(Number Number[]), explanation, points
	passingScore / retryLimit	Number	default 75 / 3
	timeLimit	Number	minutes
	randomizeQuestions / questionsToShow	Boolean/Number	
	additionalQuestions / additionalQuestionsToInclude	[Question]/Number	swap-in pool
	isMaster	Boolean	
quiz-attempts	tenantId / learnerId / courseId / assessmentId	ObjectId refs	
	answers	[QuestionAnswer subdoc]	questionIndex, questionText, options[], selectedAnswerIndex, correctAnswerIndex, isCorrect, explanation
	score / totalPoints / percentage	Number	
	passed	Boolean	
	submittedAt	Date	
	proctoringSessionId	ObjectId→QuizProctoringSession	optional

shared-content • **email-templates** • **bookmarks** • **discussions**

Collection	Key Fields
shared-content	masterCourseId→Course, tenantId→Tenant, tenantCourseId→Course, sharedAt, isActive. Unique (masterCourseId,tenantId).
email-templates	type(welcome-kit/course-completion/certificate/upgrade-invoice, unique), subject, htmlContent.
bookmarks	tenantId, userId, courseId, moduleId?, lessonId?, note. Unique (tenantId,userId,courseId).
discussions	tenantId, courseId, lessonId?, userId, parentId?(threading), content, upvotes/downvotes, upvotedBy[]/downvotedBy[], isSticky, isResolved, isDeleted, editedAt.

academic-calendar • **subjects** • **sections** (academic-config)

Collection	Key Fields
academic-calendar	tenantId, academicYear ("2025-2026"), terms[{name,startDate,endDate,isActive}], holidays[{date,name,type:national/regional/custom}], createdBy. Unique (tenantId,academicYear).
subjects	tenantId, name, isDefault. Unique (tenantId,name).
sections	tenantId, grade, name. Unique (tenantId,grade,name).

6.3 Progress, Course Assignment, Certificates & Gradebook

progress — one row per learner per course

Field	Type	Notes
tenantId / learnerId / courseId	ObjectId refs	unique (tenantId,learnerId,courseId)
currentModuleId	ObjectId→Module	
status	enum	Not Started In Progress Overdue Completed Failed
completionPercentage / scorePercentage / quizScore	Number	0–100

Field	Type	Notes
isPassed	Boolean	
startedAt / completedAt	Date	
lessonProgress	Object→jsonb	map lessonId → {completionPercentage, isCompleted, currentPosition, suspendData, scormData, quizScore, lastPageSeen, lastAccessedAt}

course-assignments

Field	Type	Notes
tenantId / courseId	ObjectId refs	
targetType	enum	USER GROUP
targetId	ObjectId	User or Group id
dueDate / isMandatory	Date / Boolean	default mandatory
assignedBy / assignedAt	ObjectId→User / Date	

certificates (templates) & certificate-issued

Collection	Field	Type	Notes
certificates (templates)	tenantId	ObjectId	null = global
	name	String	
	backgroundImageUrl / logoImageUrl / signatureImageUrl	String	S3
	designation / signatoryName	String	
	textPlacements / logoPlacement / signaturePlacement	Object→jsonb	{x,y,fontSize,color,width,height}
	selectedTenants / isActive	[ObjectId]/Boolean	
	approvalStatus	enum	pending_approval approved rejected
	submittedBy / submittedByTenantId / approvedBy / approvalDate / rejectionReason	mixed	approval workflow
certificate-issued	tenantId / learnerId / courseId	ObjectId refs	unique (tenantId,learnerId,courseId)
	certificateTemplateId	ObjectId→Certificate	
	courseName / learnerName	String	denormalized snapshot
	certificateId	String	unique public verification id
	pdfUrl / qrCodeUrl / verificationUrl	String	S3 + QR
	issuedAt / expiresAt	Date	
	isComplianceCertificate / score / passingScore / passed	mixed	

grade-records (gradebook)

Field	Type	Notes
tenantId / studentId / batchId	ObjectId refs	unique (tenantId,studentId,batchId,term)
subject / term / academicYear	String	
homeworkAverage / assessmentAverage / attendancePercent / gradePoints / overallPercentage	Number	computed
finalGrade / remarks	String	A/B/C...
gradedBy / gradedAt	ObjectId→User / Date	

6.4 School Operations

batches — teaching cohorts

Field	Type	Notes
tenantId / teacherId	ObjectId refs	
name / grade / section / subject / description / term	String	
substituteTeacherIds / studentIds	[ObjectId→User]	→ join tables
academicYear	String	YYYY-YYYY
startDate / endDate	Date	
schedule	[subdoc]	{dayOfWeek 0-6, startTime, endTime, location}
maxCapacity / creditPerClass / ratePerClass / ratePerHour	Number	
defaultMeetingProvider	enum	zoom/google_meet/jitsi/manual
classType / trialClassCount / convertedToRegular / convertedAt	mixed	regular/demo/trial
batchType / source	enum	regular one_on_one / manual auto_tutoring
tutoringRequestId	ObjectId→TutoringRequest	if auto-created
status / createdBy	enum / ObjectId	draft active archived

scheduled-classes

Field	Type	Notes
tenantId / batchId / teacherId	ObjectId refs	unique (batchId,startTime)
substituteTeacherId	ObjectId→User	
title / location / classNotes	String	
startTime / endTime	Date	
status	enum	scheduled in_progress completed cancelled rescheduled
cancellationReason / rescheduleReason	String	
rescheduledTo / rescheduledFrom	ObjectId→ScheduledClass	
rescheduleApprovalStatus	enum	pending approved rejected
meetingId	ObjectId→Meeting	
attendanceMarkedAt	Date	
isRecurring / recurringPattern	Boolean/subdoc	{frequency, endDate}

credit-packages & credit-transactions

Collection	Field	Type	Notes
credit-packages	tenantId / studentId	ObjectId refs	
	packageName	String	
	purchasedCredits / usedCredits / remainingCredits	Number	
	purchaseDate / expiresAt	Date	
	status	enum	active exhausted expired
	price / notes / allocatedBy	mixed	

Collection	Field	Type	Notes
credit-transactions	tenantId / packageId / studentId	ObjectId refs	
	transactionType	enum	purchase, deduct, refund, expire, adjustment, tutoring_hold, tutoring_hold_release, tutoring_deduction
	amount / balanceAfter	Number	+add / -deduct
	relatedClassId / relatedAttendanceId	ObjectId refs	
	notes / processedBy	mixed	

attendance

Field	Type	Notes
tenantId / scheduledClassId / batchId / studentId / markedBy	ObjectId refs	unique (scheduledClassId,studentId)
status	enum	present absent late excused
classDate / notes	Date/String	
creditDeducted / creditTransactionId	Boolean/ObjectId	
editedBy / editReason / editedAt	mixed	audit of edits

homework & homework-submissions

Collection	Field	Type	Notes
homework	tenantId / batchId / teacherId	ObjectId refs	
	title / description / instructions	String	
	attachmentUrls[] / resourceLinks[{url,label}]	arrays	
	dueDate / lateSubmissionDeadline / publishedAt	Date	
	assignedToStudentIds[]	[ObjectId→User]	
	maxScore / allowLateSubmission / latePenaltyPercent	mixed	
	status / type	enum	draft published closed / assignment quiz project reading
homework-submissions	tenantId / homeworkId / studentId	ObjectId refs	unique (homeworkId,studentId)
	attachmentUrls[] / textResponse	mixed	
	submittedAt / isLate	Date/Boolean	
	status	enum	submitted graded returned
	score / finalScore / latePenaltyApplied	Number	
	feedback / richFeedback / correctedFileUrl	String	
	rubricScores	[subdoc]	{criterion,maxScore,score,comment}

teacher-payouts

Field	Type	Notes
tenantId / teacherId	ObjectId refs	unique (teacherId,periodStart,periodEnd)
periodStart / periodEnd	Date	
completedClassIds[]	[ObjectId→ScheduledClass]	
totalClassesCompleted / ratePerClass / grossAmount / netAmount	Number	
rateType	enum	per_class/per_hour/monthly/hybrid
adjustments	[subdoc]	{type:bonus/deduction/reimbursement, amount, reason, appliedBy, appliedAt}
totalBonus / totalDeductions / nonTeachingWorkAmount	Number	

Field	Type	Notes
status	enum	draft pending approved paid disputed rejected
approvedBy / approvedAt / paidAt / paymentMethod / paymentReference / notes	mixed	
source	enum	batch tutoring

meetings & meeting-attendance

Collection	Field	Type	Notes
meetings	tenantId / scheduledClassId / hostId / createdBy	ObjectId refs	
	provider	enum	zoom/google_meet/teams/jitsi/manual
	externalMeetingId / joinUrl / hostUrl / password / title	String	
	scheduledStartTime/EndTime / actualStartTime/EndTime	Date	
	status / recordingStatus	enum	scheduled/started/ended/cancelled · pending/processing/available/expired
	recordingEnabled / recordingUrls[] / isInstant / participantCount / providerMetadata	mixed	
meeting-attendance	meetingId / userId	ObjectId refs	
	role	enum	teacher student
	joinedAt / leftAt / durationMinutes	mixed	
	deviceType / deviceInfo	mixed	desktop/mobile/tablet/unknown
	joinLeaveHistory	[subdoc]	{action:join/leave, timestamp}

Other school-ops collections

Collection	Key Fields & Enums
non-teaching-tasks	tenantId, teacherId, assignedBy, title, description, category(curriculum/content/training/meeting/other), paymentAmount, status(assigned/in_progress/completed_pending/approved/rejected), dueDate, completedAt, approvedAt/By, rejectionReason, completionNotes, hoursSpent, attachmentUrls[].
teacher-levels	tenantId, teacherId(unique), currentLevel(beginner/intermediate/lead), totalClassesTaught, attendanceScore, homeworkCompletionRate, parentFeedbackScore, classesMissed, overallScore, lastCalculatedAt, levelHistory[{month,year,level,score,calculatedAt}].
tutoring-requests	tenantId, studentId, teacherId?, subject, notes, proposedSlots[{date,startTime,endTime}], confirmedSlot, status(PENDING/ACCEPTED/REJECTED/COMPLETED), rejectionReason, teacherNotes, batchId?, scheduledClassId?, creditCostSnapshot, teacherRateSnapshot, creditHoldId→CreditTransaction.
consent-records	tenantId, studentId, parentId, consentType(VIDEO_RECORDING/DATA_PROCESSING/PHOTO_USAGE), granted, grantedAt, revokedAt, ipAddress, consentVersion, notes.
call-escalations	tenantId, scheduledClassId, teacherId, callAttempts[{attemptNumber 1-3, attemptTime, phoneNumber, callStatus(initiated/ringing/answered/no_answer/busy/failed), callDuration, callSid}], status(pending/escalating/resolved/failed), resolutionTime, teacherJoinedAt, adminNotified/At, notes.
student-reminder-calls	tenantId, scheduledClassId, studentId, batchId, callAttempts[{...same shape...}], status(pending/calling/resolved/failed), resolvedAt, notes.
teacher-lateness-reminder-log	tenantId, batchId, teacherId, lastNotifiedScheduledClassId, notifiedAt. (cron dedupe)

6.5 Communications (Messaging)

Collection	Field	Type	Notes
conversations	tenantId	ObjectId→Tenant	
	type	enum	direct group
	title / groupIcon / description	String	group only
	participants	[subdoc]	{userId, joinedAt, lastReadAt, role:admin/member, isArchived, isDeleted, deletedAt, isMuted} → join table
	lastMessageText / lastMessageAt / lastMessageBy / messageCount	mixed	denormalized
	blockedUserIds[] / createdBy	mixed	
messages	conversationId / senderId	ObjectId refs	
	text / attachmentUrls[]	mixed	
	isEdited/editedAt · isDeleted/deletedBy/deletedAt	mixed	soft delete
	isFlagged / flagReason	mixed	moderation
	replyTo / forwardedFrom	ObjectId→Message	
	reactions	[subdoc]	{emoji, userId, createdAt}

6.6 Exams & Proctoring

questions (question-bank)

Field	Type	Notes
tenantId / createdBy	ObjectId refs	
subject / topic	String	
difficulty	enum	EASY MEDIUM HARD
type	enum	MCQ, MULTI_SELECT, TRUE_FALSE, SHORT_ANSWER, LONG_ANSWER, FILL_BLANK, ONE_WORD, MATCH_COLUMN
text / imageUrl / audioUrl / explanation	String	
options	[subdoc]	{text, isCorrect}
correctAnswer	Mixed	bool / string / [string]
points / negativeMarks	Number	
tags[] / isActive / version	mixed	soft-delete via isActive

Collection	Field	Type	Notes
exams	tenantId / createdBy / batchId	ObjectId refs	
	title / description / instructions / subject	String	
	duration / totalMarks	Number	minutes
	questions	[subdoc]	{questionId→Question, points, order}
	questionSelectionMode / autoSelectRules	enum/Object	MANUAL AUTO_RANDOM
	settings	Object→jsonb	randomizeQuestions, randomizeOptions, showCorrectAnswersAfter, negativeMarkingEnabled, passingScore, allowLateSubmission, graceWindowMinutes, maxAttempts
	proctoringLevel	enum	NONE SOFT
	scheduledStartTime/EndTime / status	Date/enum	DRAFT/PUBLISHED/ACTIVE/COMPLETED/RESULTS_PUBLISHED
exam-sessions	tenantId / examId / studentId	ObjectId refs	unique (tenantId,examId,studentId)
	startedAt / endedAt / serverDeadline / lastSavedAt	Date	
	status	enum	NOT_STARTED/IN_PROGRESS/SUBMITTED/AUTO_SUBMITTED/TIMED_OUT/VOIDED
	answers	Map→jsonb	questionId → {selectedOptionIndices[], textAnswer, submittedAt}
	assignedQuestions	[subdoc]	{questionId, order} randomization snapshot
	score / totalPoints / percentage / passed	mixed	
	teacherRemarks / gradedBy / gradedAt	mixed	manual grading
	proctoringFlags	Object→jsonb	tabSwitches, fullscreenExits, copyAttempts, rightClicks, shortcutAttempts, totalFlags, severityLevel
	isReconnection / disconnectedAt / reconnectedAt	mixed	

Proctoring logs & incidents (exam + quiz)

Collection	Key Fields & Enums
proctoring-log	tenantId, sessionId→ExamSession, studentId, eventType(TAB_SWITCH, FULLSCREEN_EXIT, COPY_ATTEMPT, PASTE_ATTEMPT, RIGHT_CLICK, SHORTCUT_KEY, PRINT_ATTEMPT, BLUR, BEFOREUNLOAD), timestamp, metadata, severity(LOW/MEDIUM/HIGH).
incident-report	tenantId, sessionId, examId, reviewedBy, reviewedAt, flagSummary{...counts...,total}, disposition(PENDING/DISMISSED/CONFIRMED_VIOLATION), action(NONE/WARNING/PENALTY_APPLIED/SESSION_VOIDED), remarks.
quiz-proctoring-log	tenantId, sessionId→QuizProctoringSession, learnerId, eventType(all exam events + FACE_NO_FACE, FACE_MULTIPLE, FACE_LOOKING_AWAY, FACE_LOOKING_DOWN, FACE_EYES_CLOSED, FACE_TOO_FAR, FACE_CAMERA_ERROR), timestamp, metadata, severity.
quiz-incident-report	tenantId, sessionId, assessmentId(String), reviewedBy/At, flagSummary{...+face flags...}, disposition, action, remarks.
quiz-proctoring-session	tenantId, assessmentId, courseId, learnerId, startedAt/endedAt/serverDeadline, status(IN_PROGRESS/SUBMITTED/AUTO_SUBMITTED/VOIDED), proctoringFlags{...+face counts...}, isReconnection/disconnectedAt/reconnectedAt, selectedQuestionIndices[], selectedAdditionalIndices[], questionManifest[{pool,index}]. Unique (tenantId,assessmentId,learnerId).

6.7 Audit, Analytics & Privacy

Collection	Key Fields & Enums
activity-logs	tenantId?, userId?, type(upload/storage_threshold/course_created/course_deleted/tenant_created/tenant_updated/user_action), message, metadata, timestamp.
tenant-logs	tenantId, actionType(New Learner Invited, Course Assigned to User/Group, Course Completed, Quiz Passing Score Updated, User Activated/Deactivated, Branding Updated, Storage Requested, Quiz Reset by Manager, User Nudged by Manager, Attendance Marked by Manager, Certificate Approved by Manager, Team Report Exported by Manager), description, performedBy, metadata, ipAddress.

Collection	Key Fields & Enums
analytics-events	tenantId, userId?, eventType, eventData(jsonb), sessionId, deviceType, ipAddress, createdAt.
analytics-cache	tenantId, metricKey, data(jsonb), calculatedAt, expiresAt(TTL), filters{dateFrom,dateTo,batchId,teacherId}.
privacy-audit-log	tenantId, userId, userRole, endpoint, method, fieldsStripped[], recordsAffected, ipAddress, userAgent.

7. Complete API Route Reference (by Module)

All routes require `JwtAuthGuard` unless marked **Public**. Role restrictions via `@Roles(...)` shown in the Access column. Tenant-scoped endpoints additionally enforce `TenantGuard`.

7.1 Auth & Users

Method	Path	Access	Purpose
POST	/auth/register	SUPER/TENANT/SUB_ADMIN	Create user
POST	/auth/login	Public	Email/password login → JWT
GET	/auth/profile	Auth	Current user profile
PUT/PATCH	/auth/profile	Auth	Update profile
POST	/auth/profile/upload-photo · /presigned-url	Auth	Profile picture to S3
GET	/auth/google(/callback) · /auth/microsoft(/callback)	Public (OAuth)	Google / Microsoft SSO
POST	/auth/forgot-password · /verify-otp · /reset-password	Public	OTP password reset
POST	/auth/change-password	Auth	Change own password
POST	/auth/validate-setup-token · /setup-password	Public	First-time password setup
POST	/auth/admin-reset-password	SUPER/TENANT/SUB_ADMIN	Force-reset a user's password
POST/DEL	/auth/users/:id/link-student · link-parent · unlink-student	SUPER/TENANT/SUB_ADMIN	Parent↔student linking
POST/GET/PATCH	/users/by-ids · /users · /users/search · /users/:id (+ toggle-active, promote/revoke-subadmin)	ADMIN / Auth	User listing, update, role mgmt
POST	/bulk-upload/teachers · /students · /parents · /template/:type	SUPER/TENANT/SUB_ADMIN	CSV bulk import

7.2 Tenants, Courses & Content

Method	Path	Access	Purpose
GET	/tenants/branding/public	Public	Branding pre-login
POST/GET/PATCH/DEL	/tenants (+ /:id, /onboard, /create-admin-credentials)	SUPER_ADMIN	Tenant CRUD & onboarding
GET	/tenants/current · /current/features · /usage · /storage-check	Auth	Tenant config, features, storage
PATCH/GET	/tenants/:id/video-config · /branding · /language-config	TENANT/SUB/SUPER	Per-tenant settings
POST/GET/PUT/DEL	/courses (+ /:id, /modules, /modules/:mid, /lessons)	Auth (tenant)	Course/module/lesson CRUD
GET	/courses/master/all · /master/:id	SUPER_ADMIN	Master library
POST/GET	/assessments (+ /:id, /submit)	Auth (tenant)	Quiz CRUD & submit
POST	/scorm/parse · /extract · /validate	ADMIN/TEACHER	SCORM handling
POST	/upload/presigned-url · /tus · /welcome-kit	Auth	File uploads
POST/GET	/shared-content/push-to-all/:courseId · /tenant	ADMIN	Distribute master courses
POST/GET/PATCH/DEL	/groups (+ /:id, /members)	SUPER/TENANT/SUB_ADMIN	User groups
GET/POST	/email-templates/:type	SUPER_ADMIN	Email templates
GET/POST	/academic-config/subjects · /sections	Auth	Subjects & sections
POST/GET/PATCH/DEL	/academic-calendar (+ /current-term, /year, /:id/terms, /:id/holidays)	Auth	Academic calendar
POST	/transcription/generate-captions	SUPER/TENANT/SUB_ADMIN	Whisper captions

7.3 Progress, Assignments, Certificates, Compliance, Manager, Learner

Method	Path	Access	Purpose
POST/GET/PATCH	/progress · /progress/my · /progress/:courseId · /sync-progress	Auth (tenant)	Learner progress
PATCH/POST	/player/sync · /player/xapi-statements	Auth (tenant)	SCORM/xAPI sync
PATCH/POST/GET	/learner/sync-progress · /submit-quiz · /my-certificates · /enrolled-courses · /course-progress/:id	Auth (tenant)	Learner self-service
GET/POST/DEL	/course-assignments/courses · /assign · /my-assignments · /:id	ADMIN / Auth	Assign courses to users/groups
POST/GET/PUT/DEL	/certificates (+ /:id, /:id/generate, /issued/list)	SUPER/TENANT/SUB_ADMIN	Certificate templates & issuance
GET	/certificates/issued/:id/download · /verify/:id	Public	Download / verify certificate
GET/POST	/compliance/analytics · /nudge-users · /nudge-all	TENANT/SUB_ADMIN	Compliance & nudges
GET/POST	/manager/team-stats · /team-list · /learner-courses(+/:courseId) · /team/:id · /bulk-assign · /nudge/:id · /quiz-reset · /attendance · /export-report	MANAGER	Team management
GET	/gradebook/compute · /student/:id(/transcript) · /batch/:id/rankings	Auth	Grades, transcript, rankings
POST/GET	/consent/grant · /revoke · /student/:id · /check/:sid/type · /pending	Auth	Recording/data consents

7.4 School Operations

Method	Path	Access	Purpose
POST/GET/PATCH/DEL	/batches (+ /:id, /students, /statistics, /teacher/my-batches, /student/my-batches, /:id/unarchive, /:id/permanent)	ADMIN / TEACHER / LEARNER	Batch CRUD & enrolment
POST/GET/PATCH	/scheduling/classes/generate · /teacher/classes · /student/classes · /classes/:id(/start,/complete,/cancel,/reschedule) · /admin/session-join-timestamps	ADMIN/TEACHER/LEARNER/PARENT	Class scheduling & lifecycle
GET/POST/PATCH	/credits/my-balance · /my-transactions · /packages · /allocate · /refund · /balance/:id · /student/:id(/transactions,/balance)	ADMIN / PARENT / LEARNER	Credits
POST/GET/PATCH	/attendance/mark · /class/:id · /student/:id · /batch/:id/report · /:id	ADMIN / TEACHER	Attendance
POST/GET/PATCH/DEL	/homework (+ /:id, /publish, /close, /submit, /submissions, /stats, /teacher, /student/submissions, /submissions/:id/grade)	ADMIN/TEACHER/LEARNER/PARENT	Homework lifecycle
POST/GET/PATCH	/payouts/generate · /payouts · /:id · /:id/adjustment · /submit · /approve · /reject · /pay · /teacher	ADMIN / TEACHER	Teacher payouts
POST/GET/PATCH	/meetings (+ /instant, /:id, /start, /end, /cancel, /recording, /join, /leave, /attendance, /recordings, /live-status/:id, /webhook/zoom)	ADMIN/TEACHER/(PARENT) · webhook Public	Video meetings
POST/GET/PATCH	/non-teaching-work (+ /admin, /teacher, /:id/complete, /approve, /reject)	ADMIN / TEACHER	Non-teaching tasks
GET/POST	/teacher-levels/me · /all · /recalculate(/:id)	TEACHER / ADMIN	Teacher levels
GET/PUT	/teacher-availability/me · /all · /:teacherId	TEACHER / ADMIN	Availability slots
GET	/escalations/admin · /teacher · /student-reminders/admin · /teacher	ADMIN / TEACHER	Call escalations & reminders
GET/PATCH	/demo-analytics (+ /teacher-performance, /student-journey/:id, /convert/:batchId)	TENANT/SUB_ADMIN	Demo/trial conversion
POST/GET/PATCH	/tutoring-requests (+ /available-teachers, /student, /teacher, /admin, /:id/accept, /reject, /complete)	LEARNER/TEACHER/ADMIN	1:1 tutoring marketplace

7.5 Messaging, Exams, Proctoring, Analytics, Audit

Method	Path	Access	Purpose
GET/POST/PATCH/DEL	/messages/contacts · /conversations(+/:id,/messages,/archive,/mute,/read,/group,/participants,/block) · /msg/:id(/react,/forward,/report)	Auth + Socket.IO	Messaging (also realtime /messages namespace)
POST/GET/PUT	/question-bank (+ /:id, /bulk, /subjects, /topics, /tags, /count)	ADMIN / TEACHER	Question bank
POST/GET/PUT	/exams (+ /:id, /student, /publish, /publish-results)	ADMIN/TEACHER/LEARNER	Exam creation & publishing
POST/GET/PATCH	/exam-sessions/:examId/start · /:id/save · /submit · /status · /result · /my-session · /student/:id/results · /exam/:id/submissions · /evaluate · /void · /analytics	LEARNER/ADMIN/TEACHER/PARENT	Exam attempts & grading (+ /exams WS timer)
POST/GET/PATCH	/proctoring/:sid/event · /log · /exam/:id/incidents · /:sid/incident	LEARNER / ADMIN / TEACHER	Exam proctoring
POST/GET/PATCH	/quiz-proctoring/start · /:sid/event · /complete · /session/:aid · /log · /assessment/:id/incidents · /incidents/all · /:sid/incident · /allow-retake	LEARNER / ADMIN / SUB / MANAGER	Quiz proctoring (face + behaviour)
GET/POST	/analytics/corporate(+/learner-courses,/course/:id) · /school · /attendance-trend · /teacher-performance · /teacher-dashboard · /student/:id · /batch/:id · /financial · /arr · /engagement · /batch-utilization · /track · /export(/csv) · /comparison	ADMIN/TEACHER/PARENT	Analytics & exports
GET/POST	/audit/storage/global · /storage/tenants · /activity-logs · /upgrade-invoice/* · /email- status/:id · /tenant-audit/logs · /export-pdf	SUPER / TENANT / SUB_ADMIN	Audit & storage

8. MongoDB → PostgreSQL Migration Guidance

8.1 Type & Relationship Mapping

MongoDB / Mongoose	PostgreSQL	Guidance
<code>_id: ObjectId</code>	<code>uuid</code> (or <code>bigserial</code>)	Use <code>uuid</code> + a lookup map during ETL to preserve references; or keep the 24-char hex as <code>char(24)</code> short-term
<code>ObjectId ref→X</code>	<code>uuid FK + REFERENCES</code>	Add FK constraints & indexes; enforce <code>ON DELETE</code> rules previously implicit
Embedded array of subdocs	Child table (1:N) or <code>jsonb</code>	Normalize high-query arrays (batch.schedule, payout.adjustments, message.reactions, exam.questions, levelHistory, callAttempts). Keep low-value config blobs as <code>jsonb</code>
<code>Object</code> config blobs (featureConfig, settings, providerMetadata, lessonProgress, answers Map)	<code>jsonb</code>	Keep as <code>jsonb</code> ; add GIN indexes where filtered
<code>enum</code> string fields	Postgres <code>ENUM</code> or <code>text + CHECK</code>	All enum value lists are documented in §6 — reproduce exactly
Mongoose <code>timestamps</code>	<code>created_at / updated_at timestamptz</code>	Default <code>now()</code> ; trigger for updated_at
TTL index (otps, analytics-cache)	Scheduled cleanup job / partial index	Postgres has no TTL — use a cron <code>DELETE WHERE expires_at < now()</code>
Unique compound (tenantId,...)	<code>UNIQUE</code> constraint	Sparse-unique → partial unique index <code>WHERE col IS NOT NULL</code>

8.2 Recommended Table Groups & Normalization Decisions

- **Join tables to create** from arrays: `group_members`, `batch_students`, `batch_substitute_teachers`, `course_modules` (order), `conversation_participants`, `course_selected_tenants`, `certificate_selected_tenants`, `user_parents/children`, `exam_questions`, `payout_completed_classes`.
- **Child tables** for queried subdocs: `lessons` (from module.lessons), `payout_adjustments`, `message_reactions`, `call_attempts` (escalation + student-reminder), `teacher_level_history`, `batch_schedule`, `academic_terms`, `academic_holidays`, `homework_rubric_scores`.
- **Keep as `jsonb`**: tenant *Config blobs, exam/assessment settings, progress.lessonProgress, exam-session.answers, proctoringFlags, providerMetadata, notificationPreferences, analytics event/cache data.

8.3 Migration Sequence (suggested)

1. Stand up Postgres schema with an ORM (**Prisma** recommended for Next.js) reproducing §6 exactly, including enums & FKs.
2. Build an ETL that walks each Mongo collection, generates a UUID per `_id`, stores an old→new id map, then re-points all refs.
3. Migrate in dependency order: tenants → users → courses/modules/lessons/assessments → progress/assignments/certificates → school-ops → exams/proctoring → logs/analytics.
4. Replace Mongoose models with ORM models module-by-module; the REST contract (§7) stays identical so the frontend is unaffected.
5. Swap TTL indexes for cron cleanups; re-create unique & performance indexes noted in §6.
6. Keep Redis/Bull, S3, Socket.IO, Twilio, OpenAI, OAuth unchanged — none depend on the database engine.

Key insight: Because every endpoint is already tenant-scoped and DTO-validated, the API surface (§7) is engine-agnostic. The migration is almost entirely at the persistence layer — controllers, guards, queues, gateways and integrations port across unchanged.